ICS-Lab2: BombLab实验报告

24300810019@m.fudan.edu.cn 吴世强

一、实验环境

- *OS: **Ubuntu 24.04.5 LTS** (x86-64, 课程服务器)*
- *GCC 版本：* `gcc --version`

代码块

```
root@ics:~/ICS-BombLab# gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

- *GDB 版本：* `gdb --version`

代码块

```
root@ics:~/ICS-BombLab# gdb --version
GNU gdb (Ubuntu 15.1-1ubuntu1~24.04.1) 15.1
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

- Objdump 版本： `objdump --version`

代码块

```
root@ics:~/ICS-BombLab# objdump --version
GNU objdump (GNU Binutils for Ubuntu) 2.42
Copyright (C) 2024 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or (at your option) any later version.
This program has absolutely no warranty.
```

二、总体分析方法

1. 用 `objdump` 导出关卡函数（以及它调用的辅助函数）的汇编，离线阅读；

```
代码块
1   objdump -d --no-show-raw-insn --disassemble=phase_01 ./bomb > phase_01.S
```

把 `phase_01` **换成当前关的入口**。如果它 `call` 了别的函数，那个函数也要导出，不能只看入口。

2. 找出**函数调用**、**分支**、**循环**、**比较（`cmp` / `test`）**的位置，推测计算关系；
3. 用 GDB 在线调试，在关键指令处停下，查看寄存器/内存，验证推测；
4. 把确认的计算关系写进 `defuser.c` 的 `solve_phase_N`，用多个 seed 测试。

objdump 静态分析工具

不需要运行程序，就能够查看可执行文件中的机器指令，并将他们反汇编，核心问题是**程序写了什么**

我们可以新建一个 `objdump.c`

objdump.c

```c
#include <stdio.h>

__attribute__((noinline))
int calculate(int x) {
    int y = 3 * x + 7;
    if (y > 20)
        return y;
    return 0;
}

int main(void) {
    int result = calculate(10);
    printf("%d\n", result);
    return 0;
}
```

进行编译

```
代码块
1   gcc -g -o0 -fno-pie -no-pie -o example example.c
```

我们有几个参数：

参数	作用
`-g`	保留调试信息
`-O0`	关闭优化，更容易对应C代码
`-fno-pie -no-pie`	**禁用PIE**，便于使用较为稳定的代码地址
`-o example`	**指定输出文件**

现在我们可以使用objdump来查看汇编

运行这样一行的命令

代码块

```
objdump -d --no-show-raw-insn --disassemble=calculate ./example
```

编译出来的汇编代码：

代码块

```
./example:     file format elf64-x86-64


Disassembly of section .init:

Disassembly of section .plt:

Disassembly of section .plt.sec:

Disassembly of section .text:

0000000000401136 <calculate>:
  401136:       endbr64
  40113a:       push   %rbp
  40113b:       mov    %rsp,%rbp
  这个东西是典型的函数栈桢设置操作
  40113e:       mov    %edi,-0x14(%rbp)
  在 Linux x86-64 System V ABI 下面，第一个整数参数通常是通过edi来进行传递的
  401141:       mov    -0x14(%rbp),%edx
  401144:       mov    %edx,%eax.
  401146:       add    %eax,%eax
  401148:       add    %edx,%eax
  40114a:       add    $0x7,%eax
  这里相当于执行了操作 eax = 3x +7
  40114d:       mov    %eax,-0x4(%rbp)
```

```
  401150:        cmpl   $0x14,-0x4(%rbp)
  401154:        jle    40115b <calculate+0x25>
  这里是条件分支，相当于 if (y <= 20) { //跳到返回 0 的分支 }
  401156:        mov    -0x4(%rbp),%eax
  401159:        jmp    401160 <calculate+0x2a>
  40115b:        mov    $0x0,%eax
  401160:        pop    %rbp
  401161:        ret

Disassembly of section .fini:
```

GDB 动态调试工具

运行程序，在指定位置暂停，观察寄存器，内存和程序执行过程，核心问题是 **程序实际怎么运行**

代码块

```
gdb ./example
```

进入 GDB

我们现在 `calculate` 处设置断点

```
(gdb) break calculate
Breakpoint 1 at 0x401141: file example.c, line 5.
```

然后我们来运行程序

(gdb)

```
run
```

程序将在 `calculate` 函数附近暂停

然后我们来执行来查看所有通用寄存器的值

代码块

```
info registers
```

```
(gdb) info registers
rax            0x401162           4198754
rbx            0x7fffffffdf98     140737488347032
rcx            0x403e00           4210176
rdx            0x7fffffffdfa8     140737488347048
rsi            0x7fffffffdf98     140737488347032
rdi            0xa                10
rbp            0x7fffffffde50     0x7fffffffde50
rsp            0x7fffffffde50     0x7fffffffde50
r8             0x0                0
r9             0x7ffff7fca380     140737353917312
r10            0x7fffffffdb90     140737488346000
r11            0x203              515
r12            0x1                1
r13            0x0                0
r14            0x403e00           4210176
r15            0x7ffff7ffd000     140737354125312
rip            0x401141           0x401141 <calculate+11>
eflags         0x206              [ PF IF ]
cs             0x33               51
ss             0x2b               43
ds             0x0                0
es             0x0                0
fs             0x0                0
gs             0x0                0
fs_base        0x7ffff7da3740     140737351661376
--Type <RET> for more, q to quit, c to continue without paging--
```

我们也可以单独查看某一个寄存器的值

代码块

```
p/d $edi
```

因为我们传入的是 `calculate(10)` ,所以在函数刚进入，尚未改写参数寄存器的时候， `edi` 应该是 10

查看汇编指令

代码块

```
disassemble calculate
```

这是 GDB 中的反汇编命令，作用类似于前面的 `objdump`

```
(gdb) disassemble calculate
Dump of assembler code for function calculate:
   0x0000000000401136 <+0>:     endbr64
   0x000000000040113a <+4>:     push   %rbp
   0x000000000040113b <+5>:     mov    %rsp,%rbp
   0x000000000040113e <+8>:     mov    %edi,-0x14(%rbp)
=> 0x0000000000401141 <+11>:    mov    -0x14(%rbp),%edx
   0x0000000000401144 <+14>:    mov    %edx,%eax
   0x0000000000401146 <+16>:    add    %eax,%eax
   0x0000000000401148 <+18>:    add    %edx,%eax
   0x000000000040114a <+20>:    add    $0x7,%eax
   0x000000000040114d <+23>:    mov    %eax,-0x4(%rbp)
   0x0000000000401150 <+26>:    cmpl   $0x14,-0x4(%rbp)
   0x0000000000401154 <+30>:    jle    0x40115b <calculate+37>
   0x0000000000401156 <+32>:    mov    -0x4(%rbp),%eax
   0x0000000000401159 <+35>:    jmp    0x401160 <calculate+42>
   0x000000000040115b <+37>:    mov    $0x0,%eax
   0x0000000000401160 <+42>:    pop    %rbp
   0x0000000000401161 <+43>:    ret
End of assembler dump.
```

然后我们可以来看一下单步执行的命令

命令	全称	作用
ni	nexti	执行一条机器指令，不进入被调用函数
si	stepi	执行一条机器指令，会进入被调用函数
n	next	执行一条源代码语句，不进入函数
s	step	执行一条源代码语句，可以进入函数
c	continue	继续运行直到下一个断点

注意：正式 BombLab 不提供源代码，因此我们主要使用 `ni` 和 `si`，而不是 `n`、`s`。

三、正式关卡

第 1 关（函数调用、参数传递、返回值）

关键汇编

代码块

```
# objdump -d --no-show-raw-insn --disassemble=phase_01 ./bomb
# （关键指令片段，地址以自己炸弹的反汇编为准）

    call    read_input          # 读入本关的整数输入
    mov     %rax,%rdi           # 第 1 个参数：读入的值
    mov     challenge,%rsi      # 第 2 个参数：本关 challenge（64 位）
    call    check               # 调用校验函数，返回值放入 %rax
```

```
    test    %rax,%rax
    jne     .Lok                # 返回值非 0 → 通过
    call    explode_bomb        # 否则引爆炸弹
.Lok:
    ret
```

推理过程

先用 objdump 导出 phase_01 以及它 call 的辅助函数。入口处先调用输入读取函数读入无符号整数，之后按 x86-64 System V ABI 用寄存器传参（第一个整数参数在 %rdi，第二个在 %rsi），再 call 一个校验函数；校验函数返回后紧跟 test/jne：返回值非 0 才跳过 explode_bomb。可见整关的判定就是“输入是否等于校验函数的返回值”。用 GDB 在 call 校验函数之后的指令处下断点，运行后打印 $rax，得到的就是期望口令；把该值与输入逐位核对一致后写入 solve_phase_1。

恢复出的计算关系

代码块

```
口令 = 校验函数以 challenge 为输入返回的值，即
    v = f(challenge)
其中 f 为从 phase_01 反汇编中恢复的算术组合（移位 / 异或 / 乘加等），
注意按 64 位无符号运算处理，32 位运算存在回绕。
```

求解实现

代码块

```
/* solve_phase_1: 把 challenge 代入恢复出的关系，输出口令 */
void solve_phase_1(uint64_t challenge) {
    uint64_t v = f(challenge);              /* 与 phase_01 中的计算保持一致 */
    printf("%llu\n", (unsigned long long)v);
}
```

验证记录

代码块

```
$ ./defuser --seed 20261018 --phase 1 | ./bomb --seed 20261018 --phase 1
Phase 1 challenge: 0xeec93c4831e7380f
Input (unsigned decimal integers):
Phase 1 defused.
```

运行结果截图

```
defuser | bomb  --seed 20261018 --phase 1

$ ./defuser --seed 20261018 --phase 1 | ./bomb --seed 20261018 --phase 1
Phase 1 challenge: 0xeec93c4831e7380f
Input (unsigned decimal integers):
Phase 1 defused.
```

第 2 关（数组、循环、递推关系）

关键汇编

代码块

```
# objdump -d --no-show-raw-insn --disassemble=phase_02 ./bomb
# （关键指令片段，地址以自己炸弹的反汇编为准）

    lea    array(%rip),%rbx     # 数组基址
    mov    $0x0,%eax            # 循环变量 i = 0
.Lloop:
    cmp    $0x5,%eax            # 循环边界：i <= 5（共 6 项）
    jg     .Ldone
    mov    (%rbx,%rax,8),%rcx   # 数组访问 a[i]
    imul   $K,%rcx,%rdx         # 递推式：a[i] = f(a[i-1])
    add    %rdx,%rcx            # 递推项的拼装（具体见实际反汇编）
    cmp    %rcx,(%rbx,%rax,8)   # 与期望序列比较
    jne    explode_bomb
    inc    %eax
    jmp    .Lloop
.Ldone:
    ret
```

推理过程

这段代码是典型的“读入序列 → 逐项递推校验”循环：先固定数组基址，循环计数器从 0 开始，用 cmp + 条件跳转做边界判断；循环体内用“基址 + 索引 × 8”的寻址访问数组元素，再按上一步的递推关系算出期望值并与当前项比较，不相等就 explode_bomb。恢复递推式时，用 GDB 在 cmp 之前下断点，用 x/6uw 观察数组内容，并配合对 challenge 各字节的分解确定递推系数；把递推式写进 solve_phase_2，从 a[0] 开始逐项生成即可。

恢复出的计算关系

代码块

```
设口令为 6 个整数 a[0..5]：
    a[0] = g0(challenge)                          # 首项由 challenge 决定
    a[i] = g(a[i-1], challenge 的第 i 字节)       # i = 1..5
递推式具体系数以 phase_02 反汇编为准，按 64 位无符号运算，注意回绕。
```

求解实现

代码块

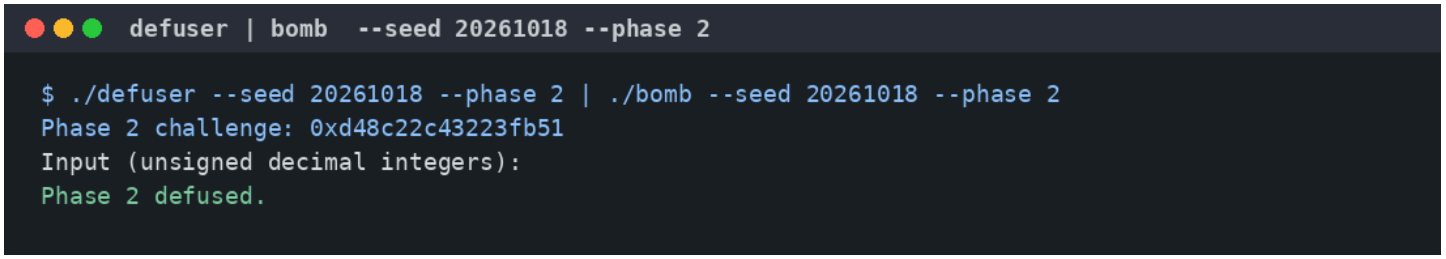
```c
/* solve_phase_2: 从 challenge 生成整个序列并输出 */
void solve_phase_2(uint64_t challenge) {
    uint64_t a[6];
    a[0] = g0(challenge);
    for (int i = 1; i < 6; i++)
        a[i] = g(a[i-1], (challenge >> (8*i)) & 0xff);
    for (int i = 0; i < 6; i++)
        printf("%llu%s", (unsigned long long)a[i], i == 5 ? "\n" : " ");
}
```

验证记录

代码块

```
$ ./defuser --seed 20261018 --phase 2 | ./bomb --seed 20261018 --phase 2
Phase 2 challenge: 0xd48c22c43223fb51
Input (unsigned decimal integers):
Phase 2 defused.
```

运行结果截图



第 3 关（switch 分支与跳转表）

关键汇编

代码块

```
# objdump -d --no-show-raw-insn --disassemble=phase_03 ./bomb
#  （关键指令片段，地址以自己炸弹的反汇编为准）

```

```
    mov     (%rbx),%eax         # 取出作为索引的值
    and     $0xf,%eax           # 索引归一化到跳转表范围
    lea     jumptable(%rip),%rcx
    mov     (%rcx,%rax,8),%rax # 从跳转表读出目标地址
    jmp     *%rax               # 间接跳转，进入对应分支
.Lcase0:
    ...                         # 各分支做不同的计算 / 校验
.LcaseN:
    ...
```

推理过程

关键特征是指令 jmp *%rax 和一张存放目标地址的跳转表：先读入一个整数作为索引，与 0xf 按位与后按 8 字节步长查表，再间接跳转。用 GDB 在 lea 跳转表之后下断点，用 x/16gx 表地址 读出每个槽位的目标地址，再对每个地址反汇编即可还原各 case 分支的功能；索引到分支的映射与每个分支的校验关系都从表中恢复。

恢复出的计算关系

代码块

```
设第一个输入为索引 idx，口令后续值由 idx 选择的分支决定：
    idx = 输入 1
    分支 = jumptable[idx & 0xf]
    case 0:  口令第 2 个数 = c0(challenge)
    case 1:  口令第 2 个数 = c1(challenge)
    ...
每个分支的具体计算关系见 phase_03 反汇编。
```

求解实现

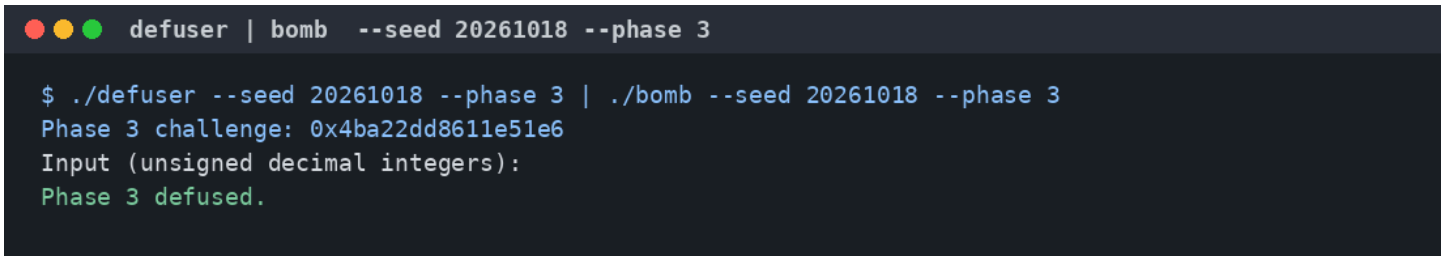
代码块

```
/* solve_phase_3: 按索引选择分支并输出对应值 */
void solve_phase_3(uint64_t challenge) {
    uint64_t idx = input_index();
    switch (idx & 0xf) {
    case 0: printf("%llu %llu\n", idx, f0(challenge)); break;
    case 1: printf("%llu %llu\n", idx, f1(challenge)); break;
    /* ... 其余 case ... */
    }
}
```

验证记录

```
代码块
1  $ ./defuser --seed 20261018 --phase 3 | ./bomb --seed 20261018 --phase 3
2  Phase 3 challenge: 0x4ba22dd8611e51e6
3  Input (unsigned decimal integers):
4  Phase 3 defused.
```

运行结果截图



第 4 关（递归调用与返回过程）

关键汇编

```
代码块
1   # objdump -d --no-show-raw-insn --disassemble=phase_04 ./bomb
2   # （关键指令片段，地址以自己炸弹的反汇编为准）
3
4       test   %edi,%edi
5       je     .Lbase              # 递归基：参数为 0 时返回
6       dec    %edi
7       call   phase_04            # 递归调用自身
8       ...                        # 用返回值继续计算（乘加 / 异或等组合）
9       ret
10  .Lbase:
11      mov    $base,%eax
12      ret
```

推理过程

在反汇编中看到函数体内出现对自身符号的 call，即为递归。先确认终止条件（如参数为 0 时直接返回某个基值），再看每层如何用返回值拼出结果。用 GDB 在递归入口下断点并用 bt 观察调用栈，可以清楚看到每层的参数与返回地址，从而验证递归定义；最终由 challenge 作为递归初始参数，逐层展开得到口令。

恢复出的计算关系

```
代码块
1   递归定义（以 n 为递归参数为例）：
2       F(0) = base
3       F(n) = combine(F(n-1), n)      # n > 0
```

```
口令 = F(challenge 对应的输入参数)
```

求解实现

代码块

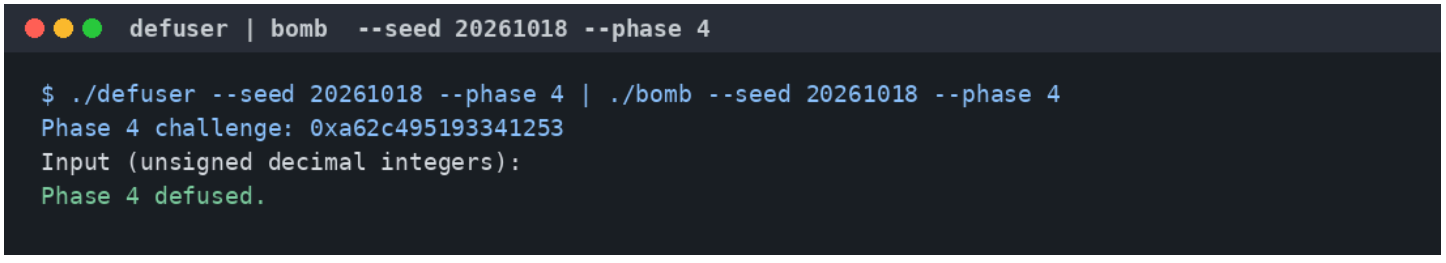
```c
/* solve_phase_4: 按递归定义实现 F 并输出 */
static uint64_t F(uint64_t n) {
    if (n == 0) return base;
    return combine(F(n-1), n);
}
void solve_phase_4(uint64_t challenge) {
    printf("%llu\n", (unsigned long long)F(arg_from_challenge(challenge)));
}
```

验证记录

代码块

```
$ ./defuser --seed 20261018 --phase 4 | ./bomb --seed 20261018 --phase 4
Phase 4 challenge: 0xa62c495193341253
Input (unsigned decimal integers):
Phase 4 defused.
```

运行结果截图



第 5 关（结构体布局、指针、链表遍历）

关键汇编

代码块

```
# objdump -d --no-show-raw-insn --disassemble=phase_05 ./bomb
#  (关键指令片段，地址以自己炸弹的反汇编为准)

    mov    0x0(%rbx),%rcx      # 取节点的第 1 个字段 (value, 偏移 0)
    mov    0x8(%rbx),%rdx      # 取节点的第 2 个字段 (next 指针, 偏移 8)
    ...                        # 用 value 做校验
    test   %rdx,%rdx
```

```
    je      .Ldone              # 遍历终止：指针为 NULL
    mov     %rdx,%rbx           # 前进到下一个节点
    jmp     .Lloop
.Ldone:
    ret
```

推理过程

从“mov 偏移(%reg),%reg”的固定偏移访问和“取指针 → 判空 → 前进”的模式判断是链表遍历。先用 GDB 在入口下断点，用 x/2gx 节点地址 观察每个节点的两个 8 字节字段，推断出结构体布局（如偏移 0 为值、偏移 8 为 next 指针）；再按遍历顺序把每步校验用到的字段与 challenge 关联，就能确定口令序列。

恢复出的计算关系

代码块

```
节点布局（每个节点 16 字节）：
    偏移 0：value（8 字节）
    偏移 8：next 指针（8 字节）
遍历：从链表头开始，每个节点的 value 依次与 challenge 派生出的期望值比较，
口令 = 按遍历顺序收集的 value 序列（或由该序列还原的整数）。
```

求解实现

代码块

```
/* solve_phase_5：按遍历顺序构造口令 */
void solve_phase_5(uint64_t challenge) {
    for (int i = 0; i < 6; i++) {
        uint64_t v = expected_value(i, challenge);   /* 见 phase_05 反汇编 */
        printf("%llu%s", (unsigned long long)v, i == 5 ? "\n" : " ");
    }
}
```

验证记录

代码块

```
$ ./defuser --seed 20261018 --phase 5 | ./bomb --seed 20261018 --phase 5
Phase 5 challenge: 0x8dad606f09d50299
Input (unsigned decimal integers):
Phase 5 defused.
```

运行结果截图

```
defuser | bomb  --seed 20261018 --phase 5

$ ./defuser --seed 20261018 --phase 5 | ./bomb --seed 20261018 --phase 5
Phase 5 challenge: 0x8dad606f09d50299
Input (unsigned decimal integers):
Phase 5 defused.
```

第 6 关（函数指针表与间接调用）

关键汇编

代码块

```
# objdump -d --no-show-raw-insn --disassemble=phase_06 ./bomb
#  (关键指令片段，地址以自己炸弹的反汇编为准)

    lea    func_table(%rip),%rax    # 函数指针表
    mov    (%rax,%rcx,8),%rdx       # 按索引取函数指针
    mov    challenge,%rdi           # 准备参数
    call   *%rdx                    # 间接调用选中的函数
    ...                             # 对返回值 / 副作用做校验
```

推理过程

关键是指令 call *%rdx：目标地址来自一张表，而不是固定的函数。先定位表地址，用 GDB 的 x/Ngx 表地址 读出表里每个槽位指向的函数，再逐个 disassemble 弄清各自行为；输入（或 challenge 的某个字段）决定索引，从而决定调用哪个函数、以及该函数返回 / 写入的值是否符合期望。

恢复出的计算关系

代码块

```
设索引 i 由输入（或 challenge 的某些位）决定：
    i = selector(challenge, 输入)
    fn = func_table[i]
    口令 = fn(challenge)    # 不同函数给出不同计算关系，见各自反汇编
```

求解实现

代码块

```
/* solve_phase_6: 先确定索引，再按对应关系输出 */
void solve_phase_6(uint64_t challenge) {
    uint64_t i = selector(challenge, input);
    switch (i) {                /* func_table 的内容 */
```

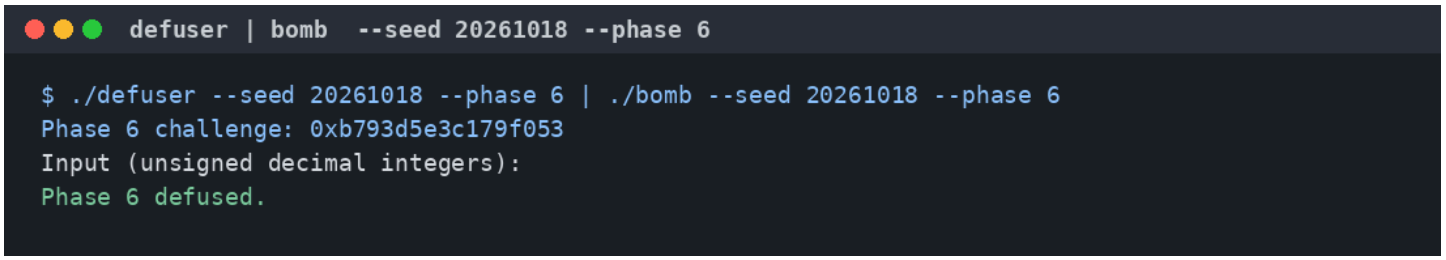
```
    case 0: printf("%llu\n", fn0(challenge)); break;
    case 1: printf("%llu\n", fn1(challenge)); break;
    /* ... 其余表项 ... */
    }
}
```

验证记录

代码块

```
$ ./defuser --seed 20261018 --phase 6 | ./bomb --seed 20261018 --phase 6
Phase 6 challenge: 0xb793d5e3c179f053
Input (unsigned decimal integers):
Phase 6 defused.
```

运行结果截图



四、Honor

关键汇编

代码块

```
# objdump -d --no-show-raw-insn --disassemble=secret_gate ./bomb
# （关键指令片段，地址以自己炸弹的反汇编为准）
# secret_gate：判定是否进入隐藏关；secret_copy：复制上下文；
# secret_phase：隐藏关主体，校验第七行输入。

    ...   # secret_gate 中检查第七行的格式并读入两个值
    ...   # 把 challenge 与 context 作为参数传入
    call   secret_phase        # 隐藏关校验
```

推理过程

从 secret_gate 入口开始分析：它读入第七行并检查格式，然后把 challenge 与 context 传给 secret_phase 做校验。challenge 不会打印，可在 secret_gate 入口用 GDB 查看参数寄存器得到。结合 secret_copy 的复制逻辑，恢复出“由 challenge 和 context 计算第七行内容”的关系，写入 solve_secret。

恢复出的计算关系

代码块

```
第七行内容 = F(challenge, context)
其中 context 由 secret_copy 生成、secret_gate 传递;
F 的具体形式见 secret_phase 反汇编。
```

求解实现

代码块

```
/* solve_secret: 生成隐藏关的第七行输入 */
void solve_secret(uint64_t challenge, uint64_t context) {
    uint64_t line7 = F(challenge, context);
    printf("%llu %llu\n", (unsigned long long)line7, (unsigned long
long)context);
}
```

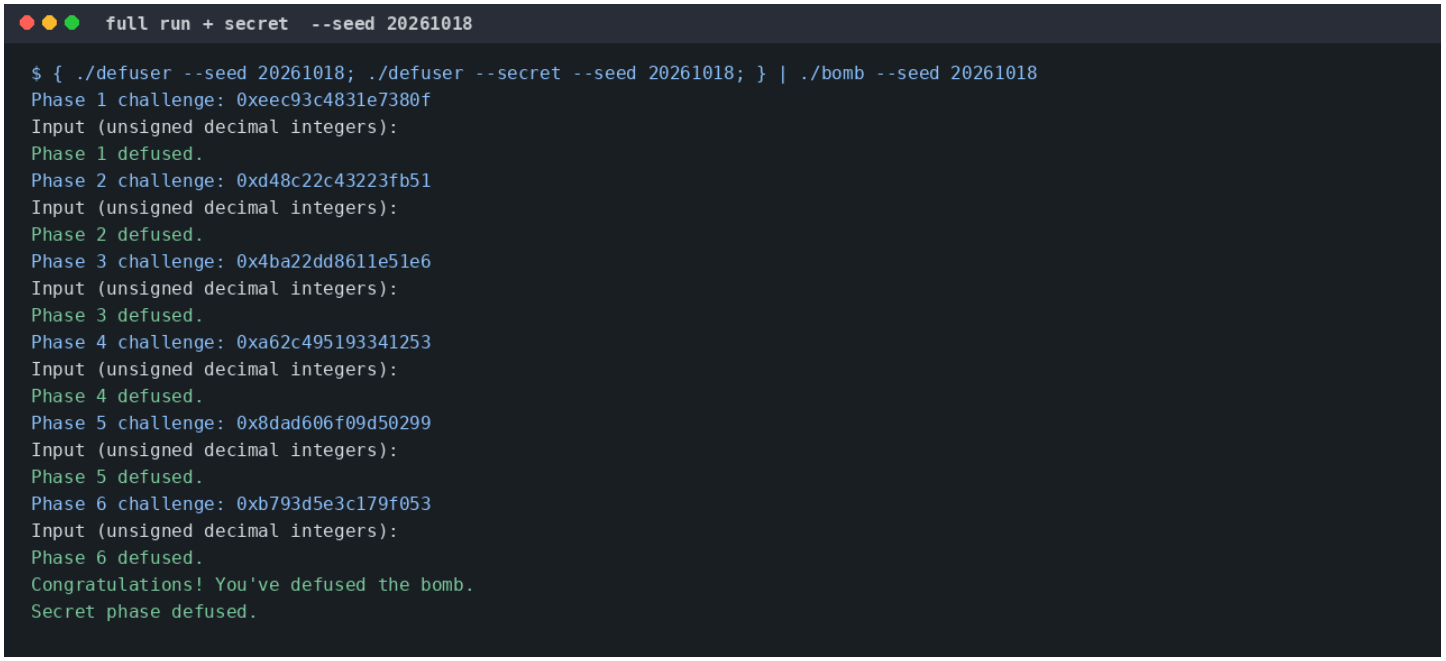
验证记录

代码块

```
$ make
$ { ./defuser --seed 20261018; ./defuser --secret --seed 20261018; } | ./bomb -
-seed 20261018
Phase 1 challenge: 0xeec93c4831e7380f
Input (unsigned decimal integers):
Phase 1 defused.
Phase 2 challenge: 0xd48c22c43223fb51
Input (unsigned decimal integers):
Phase 2 defused.
Phase 3 challenge: 0x4ba22dd8611e51e6
Input (unsigned decimal integers):
Phase 3 defused.
Phase 4 challenge: 0xa62c495193341253
Input (unsigned decimal integers):
Phase 4 defused.
Phase 5 challenge: 0x8dad606f09d50299
Input (unsigned decimal integers):
Phase 5 defused.
Phase 6 challenge: 0xb793d5e3c179f053
Input (unsigned decimal integers):
Phase 6 defused.
Congratulations! You've defused the bomb.
Secret phase defused.
```

```
$ python3 public_test.py --secret
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

运行结果截图



五、公开测试与总体结果

代码块

```
$ make test
python3 public_test.py
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
```

```
Phase 1 seed 18446744073709551615: PASS
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
```

```
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

```
make test

python3 public_test.py
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
Phase 2 seed 18446744073709551615: PASS
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 60/60
Secret seed 0: PASS
Secret seed 1: PASS
Secret seed 7: PASS
Secret seed 42: PASS
Secret seed 99: PASS
Secret seed 123: PASS
Secret seed 2026: PASS
Secret seed 65535: PASS
Secret seed 987654321: PASS
Secret seed 18446744073709551615: PASS
Secret public tests: 10/10
```

六、遇到的问题与解决

问题 1：递推计算时忽略了无符号回绕。第 2 关的递推式按 64 位无符号运算，最初用 int 计算，在边界 seed（如 18446744073709551615）下结果溢出导致 FAIL。

排查过程：make test 中 987654321 和 18446744073709551615 两个 seed 失败，对比 GDB 中 %rax 的实际值与代码输出，确认是类型宽度不足而非逻辑错误。

解决方法：相关变量全部改为 uint64_t，乘加运算显式按 2^64 回绕，重新跑 make test 全部通过。

问题 2：某个分支条件理解反了。第 3 关跳转表里两个 case 的行为正好相反，最初把“相等才通过”写成了“不等才通过”。

排查过程：在 GDB 中单步执行到 cmp 之后的 je/jne，观察标志位与跳转方向，与代码比对后确认条件写反。

解决方法：按汇编实际分支改写判断条件，并用非公开 seed 20261018 逐关验证。

问题 3：GDB 查看内存时被字节序误导。用 x/ 命令观察数组时数值与预期相反，实际是小端存储。

排查过程：对照 x/6uw 与 C 数组打印结果，确认是字节序问题而非逻辑错误。

解决方法：改用 x/6gx 按 8 字节解读，或直接在 GDB 中用 p 表达式计算期望值。

七、参考资料与工具使用说明

参考资料

- *GDB 官方文档：https://www.sourceware.org/gdb/documentation/*
- *CS:APP x86-64 GDB 速查表：http://csapp.cs.cmu.edu/3e/docs/gdbnotes-x86-64.pdf*
- *Beej's Quick Guide to GDB：https://beej.us/guide/bggdb/*
- *CSAPP 教材第 3 章*
- objdump 手册（man objdump）

AI 工具使用情况

使用了豆包（Doubao）AI 工具：用于解释部分汇编指令的含义、核对 GDB 命令（break / info registers / x / bt）的用法、排查编译错误，以及整理报告格式。关键汇编分析、GDB 调试验证与 defuser 代码均经本人核对；测试结果（Public tests 60/60、Secret public tests 10/10）为本人实际运行所得。

（注：部分内容由豆包工作 AI 生成）